

DesignForge: README & AI Usage Report

Project: DesignForge - AI-Powered LLD Practice & Review Platform | **Repository:** <https://github.com/Yogx09/DesignForge-AI-Powered-LLD-Practice-Review-Platform.git> | **Date:** September 2026

1. Quickstart & How to Run

```
# 1. Clone repository
git clone https://github.com/Yogx09/DesignForge-AI-Powered-LLD-Practice-Review-Platform.git
cd DesignForge-AI-Powered-LLD-Practice-Review-Platform

# 2. (Optional) Configure environment variables
cp .env.example .env
# Set GEMINI_API_KEY for live Google Gemini AI evaluation

# 3. Install dependencies & build
npm run build

# 4. Start backend & frontend concurrently
npm run dev
# Frontend: http://localhost:5173
# Backend API: http://localhost:4000

# 5. Run automated test suite
npm test
```

2. Environment Configuration & Hybrid Evaluator Modes

Variable	Default	Description
<code>GEMINI_API_KEY</code>	<i>Optional</i>	When set, unlocks live Gemini 1.5 Flash AI evaluation (<code>AI_GEMINI</code>) and interactive mentor chat. If omitted, automatically falls back to the deterministic AST rule engine (<code>HEURISTIC_RULE_ENGINE</code>).
<code>DATABASE_URL</code>	<i>Optional</i>	PostgreSQL connection string. If omitted, defaults to high-performance durable In-Memory store.
<code>PORT</code>	<code>4000</code>	Backend Express REST API port.

3. AI Usage Report: Key Architectural Decisions

This section details the key AI-assisted decisions made during design and implementation, documenting accepted/rejected suggestions and engineering rationale:

Decision 1: Structured Multi-Faceted Submission vs. Single Code Blob

- **AI Suggestion:** Single monolithic code editor where candidates write classes and comments in one file (similar to LeetCode).
- **Decision: Rejected** single code blob; **Accepted** multi-section structured submission (*Assumptions, Mermaid Class Diagram, Code Implementation, Design Patterns & Trade-offs*).
- **Rationale:** In real-world LLD interviews (Google, Amazon, Uber), interviewers evaluate assumptions, structural relationships, and trade-offs before judging syntax. Separating sections enables independent rubric scoring.

Decision 2: Rubric-Grounded Evaluation vs. Unconstrained "100-point AI Score"

- **AI Suggestion:** A prompt asking the LLM: *"Rate this LLD solution out of 100 and give general feedback."*
- **Decision: Rejected** generic unconstrained scoring; **Accepted** a strict 5-dimension zero-baseline rubric with JSON schema constraints and mandatory verbatim evidence quoting.
- **Rationale:** Unconstrained prompts produce inconsistent, hallucinated grades. Weighted criteria (Domain Modeling 25%, Abstraction 25%, Extensibility 20%, Edge Cases 15%, Rationale 15%) ensure reproducible, actionable feedback.

Decision 3: Deterministic Static Checks + Heuristic Fallback vs. Pure LLM Pipeline

- **AI Suggestion:** Rely solely on OpenAI/Gemini API calls for all validation, syntax checks, and rubric grading.
- **Decision: Rejected** pure LLM dependency; **Accepted** a Composite Evaluator with a fast `DeterministicValidator` and an offline `HeuristicEvaluator` fallback.
- **Rationale:** Static AST checks run in <10ms; offline heuristic fallback guarantees the platform never crashes or fails even if API keys are missing or rate-limited.

Decision 4: Version-over-Version Delta Tracking vs. Isolated Stateless Submissions

- **AI Suggestion:** Reset the attempt upon each submission or treat every submission as an isolated one-off assessment.
- **Decision: Rejected** isolated submissions; **Accepted** an `Attempt` Aggregate Root maintaining an append-only `Submission[]` history with automated delta computation (`$Attempt_{N} - Attempt_{N-1}$`).
- **Rationale:** The true value of a practice platform is the learning loop: *Design → Submit → Review → Refine → Re-evaluate*. Tracking score deltas (e.g. *" +15 on Abstraction; resolved 2 critical coupling concerns"*) reinforces architectural growth.